

Mini Project 2 · 迷你西洋棋 AI

MiniChess AI 演算法說明

從 Negamax 到 PVS 主變例搜尋引擎(含程式碼解說)

vs Weak 100%

vs Strong 100%

vs Boss 100%

PVS + 置換表 + 靜止搜尋 + 選擇性深度 | 10 局全勝(2000ms,雙方輪流先手)

OVERVIEW

整體架構:分層強化

引擎以 **Negamax + 迭代加深** 為骨架,

再一層層往上疊加:核心剪枝 → 加速結構 → 選擇性深度 → 評估函數。

每一層都讓搜尋「更快」或「更準」,
最終在相同時間內看得更深、下得更強。

評估函數

子力 · 位置表 · 王安全 · 兵形

選擇性深度

將軍延伸 · LMR · 空著裁剪 · 期望窗口

加速結構

置換表 · 走法排序 · 靜止搜尋

核心搜尋

Negamax → Alpha-Beta → PVS

Negamax 負極大值

```
if (game_state == WIN)
    return P_MAX - ply;           // 越快將死越好

int best = M_MAX;
for (auto& a : legal_actions) {
    State* nx = next_state(a);
    int score = -eval_ctx(nx, depth-1, ...);
    if (score > best) best = score; // 取最大
}
return best;
```

統一雙方視角:取最大,並對子節點取負。

- 一條公式處理雙方(零和賽局)
- $\text{score} = -\text{eval_ctx}(\text{child})$ 是核心
- 正分 = 對走子方有利
- 終端:可吃王回傳 $P_MAX - \text{ply}$
- 是 Alpha-Beta / PVS 的基礎

Alpha-Beta 剪枝

```
for (auto& a : moves) {           // 已 MVV-LVA 排序
    int score = -eval_ctx(nx, d-1,
                        -beta, -alpha, ...);

    if (score > best) best = score;
    if (score > alpha) alpha = score;
    if (alpha >= beta) break;      // beta cutoff
}
```

用 $[\alpha, \beta]$ 窗口剪掉對手不會允許的分枝。

- α : 我方已保證的下限
- β : 對手會容許的上限
- $\text{score} \geq \beta \rightarrow$ 對手不會走 $\rightarrow$ 剪枝
- 結果與完整 minimax 相同
- 最佳排序下約 $O(b^{(d/2)})$, 深度加倍
- 搭配 MVV-LVA 排序提高剪枝率

PVS 主變例搜尋

```
if (first) {                // 主變例
    score = -eval(nx, d-1, -beta, -alpha);
} else {                    // 其餘走法
    score = -eval(nx, d-1, -alpha-1, -alpha);
    // 零寬窗口意外更優 → 完整窗口重搜
    if (score > alpha && score < beta)
        score = -eval(nx, d-1, -beta, -alpha);
}
```

第一手用完整窗口,其餘用零寬窗口快篩。

- 零寬窗口 $[\alpha, \alpha+1]$: 只問「是否更好」
- 布林測試比完整搜尋快很多
- 排序良好時,重搜很少發生
- 因此比一般 alpha-beta 更快
- 實際對局採用此演算法

MOVE ORDERING

走法排序:讓剪枝有效

```
// 吃子:MOV-LVA;殺手:-1;安靜:-10
int as = av
  ? (piece_val[av]*10 - piece_val[aa])
  : (a_kill ? -1 : -10);

// 根節點先試置換表走法
rotate(moves, tt_move);
```

好棋先試,剪枝才會有效。

- 順序:置換表手 → 吃子 → 殺手 → 安靜
- MOV-LVA:吃大子、用小子吃
- 殺手:曾造成 cutoff 的安靜走法
- 置換表手:前一層算出的最佳手
- 好棋先試 → 更早 cutoff → 看更深

QUIESCENCE

靜止搜尋:避免視界效應

```
int stand_pat = evaluate(...);
if (stand_pat >= beta) return beta;
if (stand_pat > alpha) alpha = stand_pat;

// delta 剪枝:吃了也拉不回 alpha 就跳過
if (stand_pat + capture_val[cap] + 50
    < alpha) continue;

// 只搜吃子,直到局面安靜
```

葉節點只追吃子到安靜才評估。

- 視界效應:停在被吃子前會誤判佔優
- stand-pat:「不吃」的評估基準
- delta pruning:跳過無望的吃子
- 上限 MAX_QDEPTH = 8
- 讓葉節點的評估在戰術上可靠

置換表 與 Zobrist 雜湊

```
// 探測:同局面直接重用
TTEEntry& e = g_tt[key & TT_MASK];
if (e.key == key && e.depth >= depth) {
    if (e.flag == TT_EXACT) return e.score;
}

// Zobrist 增量更新(走子時)
h ^= zob[p][orig][from]; // 移出
h ^= zob[p][moved][to];  // 移入
```

快取局面結果;雜湊 $O(1)$ 增量更新。

- 100 萬筆($TT_BITS = 20$)雜湊表
- key 驗證碰撞;深度優先替換
- 存深度、分數、界限、最佳走法
- Zobrist:每(子,格,色)一隨機數
- 將死分數依 ply 正規化

SELECTIVE DEPTH

選擇性深度:延伸 / 縮減 / 裁剪

```
// 將軍延伸:強制應對多搜 1 層
int ext = (ply<20 && nx->is_king_attacked())
        ? 1 : 0;

// 後期走法縮減 (LMR)
if (!ext && depth>=3 && move_index>=3
    && !capture) reduction = 1;

// 空著裁剪
if (null_score >= beta) return beta;
```

把搜尋深度花在關鍵的著法上。

- 將軍延伸:被將軍的線多搜 1 層
- LMR:後段安靜手先搜淺,意外才重搜
- 空著裁剪:跳一手仍 $\geq \beta$ 就剪枝
- 期望窗口:窄窗口 ± 50 提高剪枝率
- 皆為啟發式,設條件控制風險

EVALUATION

評估函數:為局面打分

```
self_score += kp_material[piece];           // 材料
self_score += pst[piece-1][row][col];       // 位置表
self_score += king_tropism(...);           // 王接近
// + 通路兵 / 車開放線 / 兵形罰分

bonus += 3 * (self_mob - oppn_mob);

return self_score - oppn_score + bonus;
```

分數 = 我方總分 - 對方總分(走子方視角)。

- 材料:兵20 馬60 象65 車100 后200
- 位置表 PST:每子每格的位置加分
- 王接近度:子力靠近敵王給壓力
- 通路兵 / 車開放線 / 兵形(疊兵孤兵扣分)
- 機動力:可走步數差 $\times 3$

THE KEY FIX

關鍵修復:根節點存入置換表

```
// 每層結束:把根節點最佳手寫入 TT
if (best_move != Move{} && !ctx.stop) {
    TTEntry& rt = g_tt[root_key & TT_MASK];
    rt.key      = root_key;
    rt.best_move = pack_move(best_move);
    rt.depth    = depth;
    rt.flag     = TT_EXACT;
}
```

結果:Weak 40% → 100%

限時被中斷時,不再走出敗著。

- 舊版:每層用吃子排序,把「會輸的吃子」排第一
- 被 kill 取最後回報手 → 開局第 7 手被將死
- 新版:每層先搜、先回報真正最佳手
- 固定深度本來就對 → 補上「限時」這條路

RESULTS

成果:對三個對手全勝

測試設定:每步 2000ms、各 10 局、雙方輪流先手

vs Weak

100%

10 勝 0 敗

修正前 40% → 100%

vs Strong

100%

10 勝 0 敗

修正前 90% → 100%

vs Boss

100%

10 勝 0 敗

修正前 70% → 100%

關鍵修正(根節點存入 TT)讓三個對手皆達穩定全勝。

SUMMARY

總結

這是一個以 Negamax 為基礎、採用迭代加深的 PVS 引擎:

alpha-beta + PVS 為核心,搭配置換表與 MVV-LVA/殺手排序讓剪枝有效,

靜止搜尋避免戰術失誤,再以延伸/縮減/裁剪把深度花在關鍵處,餵給完整評估函數。

PVS 架構

比 alpha-beta 更快收斂

置換表 + 排序

讓搜尋看得更深

根節點存 TT

修掉限時敗著:40%→100%

謝謝聆聽 · Weak / Strong / Boss 全數 10:0